

Practical Exam: SWD392 -SU25

Duration: 85 Minutes

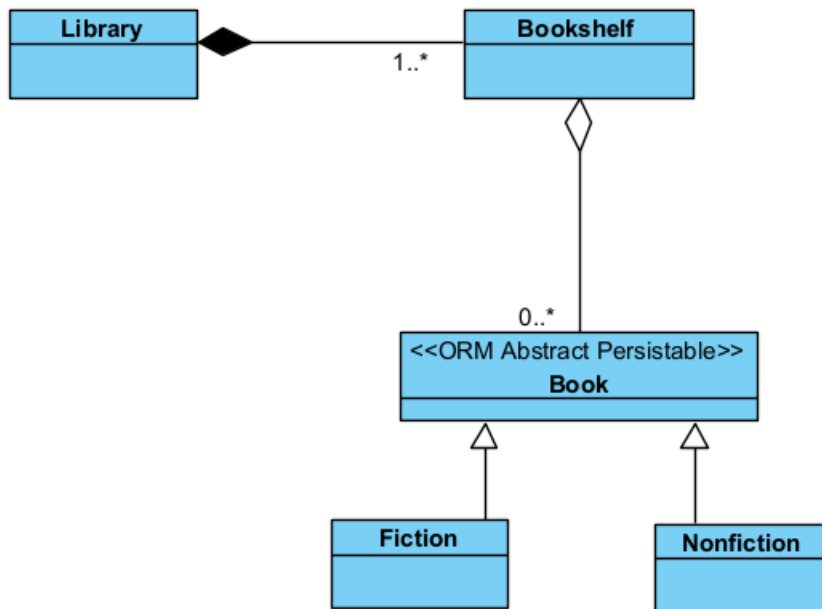
Instructions:

- You are a software engineer at "Cyber AI FPTx" company. Your project is to build a new **Library Management System** for the university.
 - Use your knowledge and provide real-world assumptions where needed.
 - Draw all diagrams using a UML tool. Submit your diagrams and written answers in a single document.
 - The language level of this exam is designed to be clear and simple.
-

Questions 1. (3.0 points)

Business Scenario: The system needs to model how books are organized. A Library contains many Bookshelves. Each Bookshelf is a physical part of the library and cannot exist without it. A Book is placed on exactly one Bookshelf. Books can be either Fiction or NonFiction types, which have different attributes for categorization.

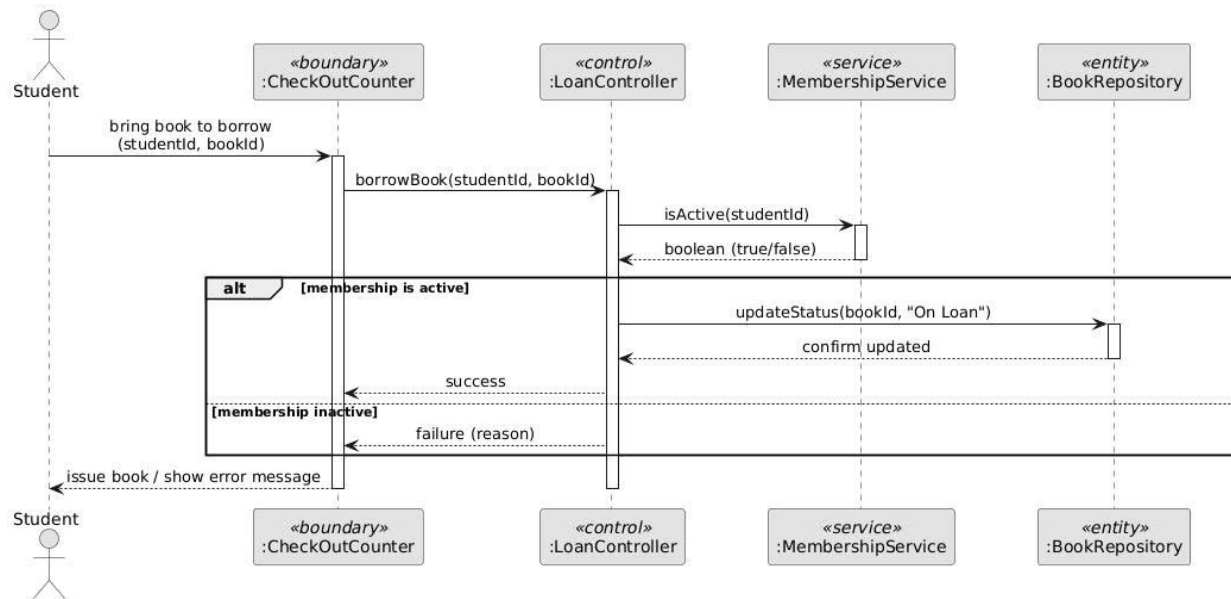
- **Task:** Draw a **Class Diagram** at the entity level (showing only class names, no attributes).
 - Your diagram must show the relationships between Library, Bookshelf, Book, Fiction, and NonFiction.
 - You must correctly show **aggregation**, **composition**, and **generalization** relationships, including multiplicities.



Questions 2. (3.0 points)

Business Scenario: A student wants to borrow a book. The student brings the book to the CheckOutCounter (a boundary object). The counter staff uses the system, which calls a LoanController (a control object). The controller first checks the student's status with a MembershipService (a service object) to see if their account is active. Then, it updates the book's status via a BookRepository (an entity object) to mark it as "On Loan".

- **Task:** Choose the **best** UML diagram (Sequence or Communication) to clearly show the **step-by-step, time-ordered flow of messages** for this process. Draw the diagram you have chosen. **You must label each object with its correct application logic stereotype («boundary», «control», etc.).**



Questions 3. (2.0 points)

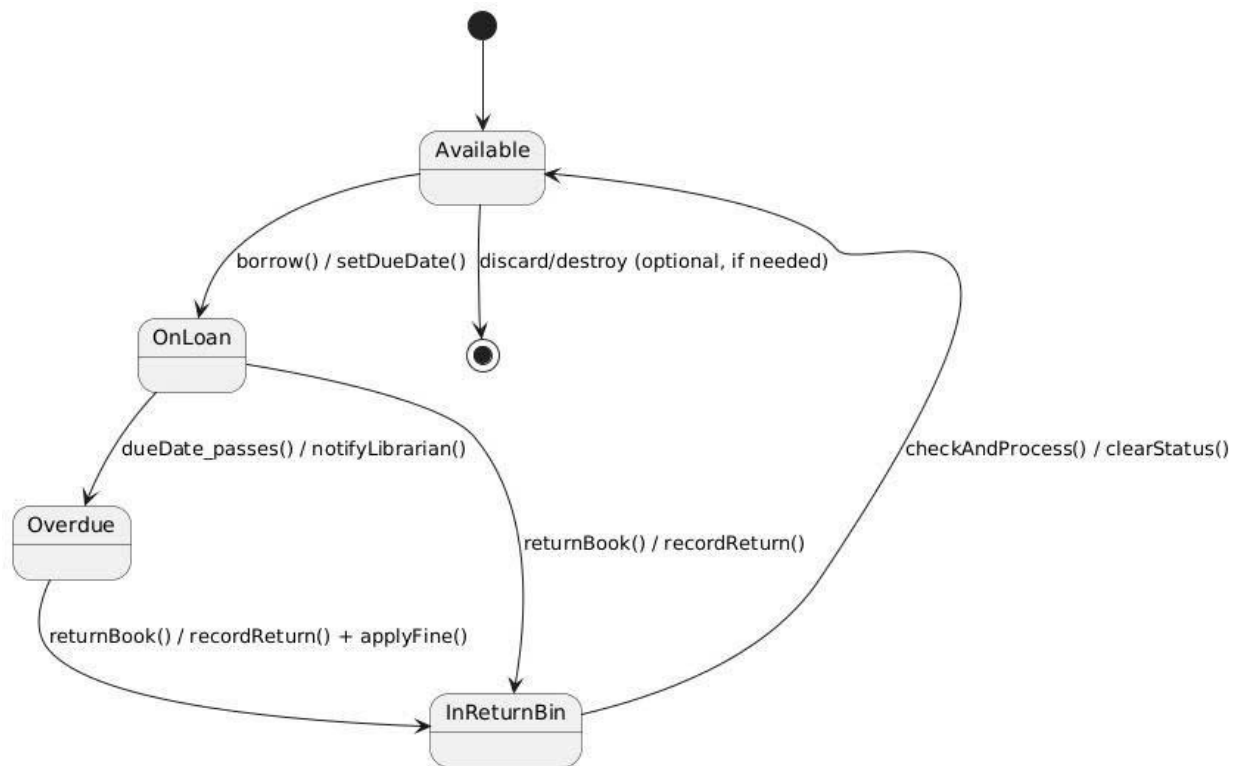
Business Scenario: A physical book in the library has a lifecycle. It starts in the Available state. When a student borrows it, it transitions to the OnLoan state. If it is not returned on time, an event `dueDate_passes` occurs, and it enters the Overdue state. When the student returns it, it goes into the InReturnBin state for processing. After a librarian checks it, it goes back to the Available state.

- **Task:** Draw a **Statechart Diagram** for a Book object.
 - Your diagram must show all the states: Available, OnLoan, Overdue, InReturnBin.
 - Clearly label the **events** that trigger the state changes and the **actions** that occur.

Questions 4. (1.0 point)

Task: Propose a suitable software architecture for the Library Management System.

- Name the architecture you choose.
- **Explain why this architecture is a good choice for this project.** Then, list one key advantage and one key disadvantage.



Questions 5. (1.0 point)

Business Scenario: The system allows users to search for books. Users can search by Title, by Author, or by ISBN. Each search type requires a different search algorithm. The main search interface should allow the user to select one of these search strategies and execute it without the interface itself knowing the details of the search algorithm.

- **Task:** Identify and name a specific **Design Pattern** that allows you to define a family of algorithms, encapsulate each one, and make them interchangeable. **Explain why this pattern is the best solution for this specific problem.**

The design pattern that perfectly solves this problem is the Strategy Pattern.

Why Strategy Pattern is the exact match

The Strategy pattern is defined as: *"Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it."*

This matches the scenario word-for-word:

- **Family of algorithms** → Search by Title, Search by Author, Search by ISBN (each with its own completely different implementation, e.g., different indexing, fuzzy matching, or database queries).
- **Encapsulate each one** → Each search algorithm lives in its own class (e.g., TitleSearchStrategy, AuthorSearchStrategy, IsbnSearchStrategy).

- **Make them interchangeable** → The main search interface (the Context) can swap strategies at runtime without any code changes.

END OF EXAMINATION

(Invigilators will not provide further explanations)

